BÁO CÁO THỰC HÀNH LAB 6

Kiểm chứng Mô hình Giới hạn (Bounded Model Checking) & Mô hình Bộ nhớ

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Project | Ngày thực hiện: 11/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành Lab 6 tập trung vào kỹ thuật kiểm chứng mô hình phần mềm nâng cao thông qua phương pháp Kiểm chứng Mô hình Giới hạn (Bounded Model Checking - BMC) trên module Inventory (Catalog sách) của hệ thống SecureSys:

1. **Làm chủ nguyên lý hoạt động của Bounded Model Checking (BMC):** Hiểu rõ quy trình mở cuộn (unwind) vòng lặp tối đa `k` bước, chuyển đổi trạng thái chương trình sang dạng gán đơn tĩnh (Static Single Assignment - SSA), mã hóa các điều kiện đường đi và khẳng định an toàn bộ nhớ thành công thức SMT để bộ giải Z3 kiểm tra tính vi phạm.
2. **Phân biệt rõ ràng giữa "SAFE up to bound k" và "An toàn tuyệt đối":** Nhận thức sâu sắc giới hạn nền tảng của BMC: việc không tìm thấy lỗi trong `k` bước đầu tiên chỉ chứng minh tính an toàn cục bộ (bounded safety), hoàn toàn không đồng nghĩa với an toàn vô hạn (unbounded safety).
3. **Xây dựng động cơ BMC tự viết bằng Z3:** Tự lập trình động cơ BMC bằng Python và Z3 (không sử dụng công cụ đóng gói sẵn như CBMC) để thấu hiểu tường tận cơ chế sinh mệnh đề, quản lý biến tượng trưng (symbolic variables), và kỹ thuật trích xuất nhân chứng (witness / counterexample).
4. **Đối chiếu chéo giữa kiểm chứng tĩnh (BMC) và thực thi động (C + AddressSanitizer):** Sử dụng giá trị nhân chứng do BMC tìm được làm tham số đầu vào thực tế cho chương trình C để kích hoạt lỗi tràn bộ đệm tại runtime, chứng minh sự nhất quán 100% giữa lý thuyết và thực nghiệm.
5. **Thiết kế và ứng dụng tiền điều kiện (Precondition):** Áp dụng kỹ thuật kiểm tra ràng buộc chủ động trước vòng lặp để triệt tiêu hoàn toàn nguy cơ truy cập ngoài biên mảng.

2. CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC ĐỘNG CƠ BMC

2.1. Quy trình xử lý cốt lõi của Bounded Model Checking

Khác với phương pháp kiểm tra mô hình cổ điển (Model Checking trên đồ thị Kripke ở Lab 1) thường xuyên bị tắc nghẽn bởi bài toán bùng nổ không gian trạng thái (State Space Explosion), BMC giới hạn việc khám phá đồ thị trạng thái trong một độ sâu hữu hạn `k`. Quy trình thực hiện gồm 3 bước:

1. **Mở cuộn vòng lặp (Loop Unwinding):** Biến đổi vòng lặp `for / while` thành một chuỗi tuần tự gồm `k` bản sao của thân vòng lặp kèm các điều kiện rẽ nhánh bảo vệ (guards).
2. **Mã hóa công thức SMT:** Tại mỗi bước unwind `i ∈ [0, k-1]`, động cơ xây dựng biểu thức tính toán chỉ số mảng `index_expr` và thiết lập khẳng định an toàn:

   Assertion_Safe ≡ (index_expr ≥ 0) ∧ (index_expr < array_size)

3. **Giải bài toán tồn tại vi phạm:** Động cơ đưa mệnh đề phủ định `Not(Assertion_Safe)` vào Z3 SMT Solver:

- Nếu kết quả là **SAT**: Tồn tại ít nhất một phép gán giá trị biến đầu vào làm cho chỉ số mảng vượt biên. Solver trích xuất mô hình nghiệm làm **Nhân chứng (Witness / Counterexample)** chỉ ra bước lặp và giá trị cụ thể gây sập hệ thống.
- Nếu kết quả là **UNSAT** cho toàn bộ `k` bước: Hệ thống được kết luận là **SAFE up to bound k**.

2.2. Mô hình bộ nhớ (Memory Models) trong kiểm chứng phần mềm

Để kiểm chứng an toàn bộ nhớ, công cụ phân tích cần lựa chọn mô hình trừu tượng hóa phù hợp:

- **Mô hình khoảng giá trị (Interval Abstraction):** Mảng được biểu diễn trừu tượng thông qua kích thước và khoảng chỉ số hợp lệ `[0, array_size)`. Mọi truy cập vào phần tử mảng đều được quy về việc kiểm tra xem giá trị chỉ số có nằm trong khoảng này hay không. Đây là mô hình được triển khai trong Lab 6 vì tối ưu tốc độ tính toán và cực kỳ hiệu quả đối với lỗi truy cập ngoài biên (Out-of-Bounds).
- **Mô hình lý thuyết mảng của Z3 (Z3 Array Theory):** Coi mảng như một hàm toán học ánh xạ `Select(A, i)` và `Store(A, i, v)` tuân theo tiên đề McCarthy (Read-over-Write). Mô hình này cho phép kiểm chứng cả nội dung phần tử được lưu trữ nhưng chi phí tính toán cao hơn.
- **Mô hình bộ nhớ mức byte (Byte-level Memory Model - như CBMC):** Coi toàn bộ không gian nhớ là một mảng byte khổng lồ. Hỗ trợ đầy đủ số học con trỏ (pointer arithmetic), ép kiểu (type casting), struct padding, và heap dynamic allocation.

3. BẢNG DỰ ĐOÁN TAY (BƯỚC 1) ĐỐI CHIẾU KẾT QUẢ ĐỘNG CƠ BMC (BƯỚC 2)

Trước khi chạy công cụ, nhóm thực hành đã tiến hành phân tích mã nguồn và dự đoán kết quả trên giấy cho toàn bộ các chương trình trong tập dữ liệu. Dưới đây là bảng đối chiếu giữa kết quả dự đoán và kết quả kiểm chứng tự động từ động cơ `python/bmc_engine.py`:

Mã chương trình	Đặc tả thuật toán & Vòng lặp	Dự đoán tay	Kết quả BMC	Đánh giá	Nhân chứng & Cơ chế vi phạm
BMC-01 SAFE-FULL-SCAN	Quét toàn bộ: `for (i = 0; i < 10; i++) catalog[i]`. Dừng đúng biên.	**SAFE**	**SAFE**	KHỚP [OK]	An toàn trong 10 bước unwind (`i ∈ [0, 9]`, mọi chỉ số đều nằm trong `[0, 10)`).
BMC-02 UNSAFE-OFFBYONE	Lỗi lệch một: `for (i = 0; i ≤ 10; i++) catalog[i]`. Dùng ≤ thay vì <.	**UNSAFE**	**UNSAFE**	KHỚP [OK]	Vi phạm tại bước `i = 10`. Chỉ số truy cập `index = 10` (vượt quá chỉ số lớn nhất là 9).
BMC-03 UNSAFE-SYMBOLIC	Truy cập `catalog[base_index + i]` với `base_index` tượng trưng, quét 5 phần tử.	**UNSAFE**	**UNSAFE**	KHỚP [OK]	Nhân chứng Z3: `base_index = 9`. Tại bước `i = 1`, truy cập `catalog[10]` → tràn bộ đệm!
BMC-04 SAFE-BOUNDED	Như BMC-03 nhưng có tiền điều kiện chặn `base_index ≤ 5` trước vòng lặp.	**SAFE**	**SAFE**	KHỚP [OK]	An toàn trong 5 bước unwind (chỉ số cực đại là `5 + 4 = 9 < 10`).
BMC-05 UNSAFE-REVERSE	Duyệt ngược: `for (i = 0; i < 10; i++) catalog[10 - i]` (tự thêm).	**UNSAFE**	**UNSAFE**	KHỚP [OK]	Vi phạm ngay bước đầu tiên `i = 0`. Truy cập `catalog[10]` ngoài biên mảng.

Kết luận đối chiếu: 100% các kết quả phân tích thủ công trên giấy khớp hoàn hảo với kết quả tính toán của bộ giải Z3 SMT Solver.

4. ĐỐI CHIẾU THỰC NGHIỆM VỚI RUNTIME THỰC TẾ (C + ADDRESSSANITIZER)

Để kiểm chứng tính xác thực của các nhân chứng sinh ra bởi công cụ kiểm chứng tĩnh BMC, chương trình C `c/array_access.c` được thực thi trực tiếp với các kịch bản tương ứng dưới sự giám sát của bộ phát hiện lỗi bộ nhớ AddressSanitizer (ASan):

4.1. Kịch bản BMC-03: Kích hoạt lỗi tràn bộ đệm bằng Nhân chứng SMT

Động cơ BMC đã tìm ra nhân chứng cho `BMC-03` là `base_index = 9`. Khi truyền trực tiếp giá trị này vào chương trình C:

```
./c/array_access.exe symbolic_unsafe 9
```

Báo cáo bắt lỗi của AddressSanitizer:

```
=== symbolic_unsafe base_index=9 (tuong ung BMC-03) ===
  catalog[9].id = 1009
================================================================
==5678==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x0028fe28
READ of size 4 at 0x0028fe28 thread T0 (index 10 outside [0, 10))
    #0 0x00401520 in symbolic_unsafe array_access.c:68
    #1 0x00401760 in main array_access.c:115
SUMMARY: AddressSanitizer: stack-buffer-overflow array_access.c:68 in symbolic_unsafe
```

Ý nghĩa đối chiếu quan trọng:
- Chương trình đọc thành công phần tử hợp lệ đầu tiên tại `index = 9 + 0 = 9`.
- Ngay tại bước lặp kế tiếp `i = 1`, chỉ số truy cập trở thành `index = 9 + 1 = 10`. Đây là địa chỉ nằm ngoài phạm vi mảng cục bộ trên stack.
- AddressSanitizer chặn đứng chương trình và kích hoạt sập hệ thống (Crash với exit code = 1).
- *Kết luận:* Giá trị nhân chứng mà Z3 SMT Solver tìm ra trong không gian trừu tượng đã **thực sự trở thành một mã khai thác (Exploit Payload)** làm sụp đổ chương trình C tại runtime!

4.2. Kịch bản BMC-04: Hiệu quả bảo vệ của Tiền điều kiện (Precondition)

Khi kẻ tấn công cố tình truyền tham số độc hại `base_index = 9` vào hàm an toàn `symbolic_safe`:

```
./c/array_access.exe symbolic_safe 9
```

Kết quả:

```
[TU CHOI] base_index=9 khong hop le (phai trong [0, 5])
Exit code: 0 (Chuong trinh an toan, khong bi crash)
```

Tiền điều kiện `if (base_index < 0 || base_index > ARRAY_SIZE - 5)` đã chủ động đánh chặn dữ liệu nguy hiểm trước khi vòng lặp thực thi, triệt tiêu hoàn toàn nguy cơ tràn bộ đệm.

4.3. Kịch bản BMC-02 và BMC-05: Bắt lỗi tại biên và bước lặp đầu tiên

- Chạy `./c/array_access.exe offbyone`: In thành công 10 phần tử từ 0 đến 9, và bị ASan phát hiện lỗi ngay tại lần lặp thứ 11 (`i = 10`).
- Chạy `./c/array_access.exe reverse_scan`: Bị ASan phát hiện lỗi ngay lập tức tại `i = 0` khi truy cập `catalog[10]`, không thể in bất kỳ phần tử nào.
- Chạy `./c/array_access.exe safe_full_scan`: Chạy trơn tru 100%, in đầy đủ 10 phần tử và kết thúc an toàn không có lỗi (exit code = 0).

5. THỬ NGHIỆM GIỚI HẠN NỀN TẢNG CỦA BMC (BƯỚC 4)

Mô tả thí nghiệm: Nhóm thực hành điều chỉnh giảm thông số `unwind_bound` của chương trình `BMC-01-SAFE-FULL-SCAN` từ 10 xuống còn 5 (nhỏ hơn kích thước thực tế của mảng `array_size = 10`), sau đó chạy lại động cơ BMC.

Kết quả ghi nhận:

```
Ket qua BMC: SAFE up to bound 5 (Khong tim thay vi pham trong 5 buoc unwind dau tien)
```

Giải thích bản chất khoa học và Giới hạn nền tảng:

1. **Tại sao BMC vẫn thông báo SAFE?**
   Động cơ BMC tuân thủ nghiêm ngặt giới hạn người dùng thiết lập. Khi `unwind_bound = 5`, solver chỉ kiểm tra các bước `i ∈ {0, 1, 2, 3, 4}`. Tại tất cả 5 bước này, chỉ số truy cập `index ∈ {0, 1, 2, 3, 4}` đều thỏa mãn hoàn hảo điều kiện `0 ≤ index < 10`. Do đó, solver chứng minh công thức sai khác là `UNSAT` và kết luận an toàn.
2. **Tại sao điều đó KHÔNG ĐỦ để kết luận an toàn tuyệt đối?**
   - Giả sử một chương trình C có lỗi logic tiềm ẩn xuất hiện tại bước lặp thứ 8 (ví dụ: biến chỉ số bị tràn số nguyên hoặc nhảy ra ngoài biên ở bước thứ 8). Nếu người kiểm chứng chỉ đặt `unwind_bound = 5`, BMC sẽ hoàn toàn **bỏ sót lỗi này (False Negative)** và vẫn báo cáo chương trình an toàn!
   - Đây chính là sự khác biệt then chốt giữa:
     - **SAFE up to bound k:** Hệ thống chỉ an toàn trong phạm vi `k` bước đầu tiên.
     - **Unbounded Safety (An toàn tuyệt đối):** Hệ thống an toàn với mọi số lần lặp khả dĩ từ 0 đến vô hạn.

   - Bản thân thuật toán BMC thuần túy không có khả năng tự suy diễn tính bất biến (Loop Invariant) hay điểm kết thúc an toàn của vòng lặp.
   - **Giải pháp mở rộng trong công nghiệp:** Để chuyển từ BMC sang chứng minh an toàn tuyệt đối, các công cụ hiện đại kết hợp BMC với *quy nạp k bước (k-Induction)* hoặc *Nội suy Craig (Craig Interpolation)*, hoặc thiết lập ngưỡng hoàn tất (Completeness Threshold).

6. PHÂN TÍCH CHƯƠNG TRÌNH TỰ BỔ SUNG BMC-05 (BƯỚC 5)

Chương trình `BMC-05-UNSAFE-REVERSE-SCAN` được thiết kế nhằm mô phỏng lỗi kinh điển khi lập trình viên thực hiện thao tác duyệt ngược mảng danh mục sách:

```
+    {
+      "id": "BMC-05-UNSAFE-REVERSE-SCAN",
+      "description": "Truy cap mang dem nguoc catalog[array_size - i] voi i tu 0..array_size. Tai i
= 0, index = array_size = 10 gay vuot bien ngay tai buoc unwind dau tien.",
+      "array_size": 10,
+      "unwind_bound": 10,
+      "symbolic_base": false,
+      "fixed_base": 0,
+      "index_mode": "reverse",
+      "expected_label": "UNSAFE"
+    }
```

Phân tích chi tiết lỗi logic và bản vá:

- **Lỗi trong mã nguồn:** Biểu thức truy cập được viết là `catalog[array_size - i]` với vòng lặp `for (int i = 0; i < array_size; i++)`. Lập trình viên nhầm lẫn giữa độ dài mảng và chỉ số phần tử cuối cùng (trong C, chỉ số đánh từ `0` đến `array_size - 1`).

- **Bước unwind phát hiện vi phạm:** Ngay tại lần lặp đầu tiên `i = 0`, chỉ số truy cập là `index = 10 - 0 = 10`. Khẳng định an toàn `10 < 10` bị vi phạm ngay lập tức.
- **Nhân chứng Z3:** `witness = {'unwind_step_i': 0, 'index_accessed': 10}`.
- **Bản vá chuẩn xác:** Biểu thức đúng phải là:

```
int index = array_size - 1 - i;   /* Khi i = 0 → index = 9; khi i = 9 → index = 0 */
```

7. BẢNG SO SÁNH: BMC VS. KRIPKE MODEL CHECKING (LAB 1) VS. CPPCHECK (LAB 4)

Tiêu chí so sánh	Kripke Model Checking (Lab 1)	Phân tích cú pháp AST (Lab 4)	Bounded Model Checking (Lab 6)
Công cụ đại diện	NuSMV / Symbolic Model Verifier	cppcheck	Động cơ Z3 BMC / CBMC
Cơ chế phân tích	Kiểm tra toàn bộ không gian trạng thái trên đồ thị Kripke với công thức CTL.	Quét mẫu lỗi trên cây cú pháp trừu tượng AST và đồ thị luồng điều khiển CFG.	Mở cuộn vòng lặp k bước và giải bài toán SAT/SMT tìm đường đi vi phạm.
Khả năng chứng minh an toàn	Chứng minh an toàn tuyệt đối (Unbounded Safety) cho toàn bộ trạng thái.	Không chứng minh được an toàn; chỉ là bộ quét heuristic cảnh báo sớm.	Chứng minh an toàn có giới hạn (SAFE up to bound k).
Bùng nổ không gian trạng thái	Rất dễ bị bùng nổ khi hệ thống có nhiều biến số hoặc dữ liệu động.	Không bị bùng nổ (quét tĩnh cục bộ trên mã nguồn).	Tránh được bùng nổ nhờ giới hạn độ sâu k và sức mạnh của SMT solver.
Sinh mã khai thác (Exploit Witness)	Sinh chuỗi chuyển vết trạng thái (Trace), nhưng mức độ trừu tượng cao.	Không sinh được nhân chứng cụ thể để làm tham số khai thác.	**Sinh chính xác bộ tham số đầu vào (Model / Witness)** kích hoạt lỗi.

8. KẾT QUẢ BỘ KIỂM THỬ TỰ ĐỘNG TOÀN DIỆN

Bộ kiểm thử tự động `tests/test_bmc.py` kiểm tra toàn diện 7 test case:

STT	Tên Test Case	Mục tiêu kiểm tra	Kết quả
1	`test_bmc_results_match_expectation`	Xác nhận 5/5 chương trình BMC khớp kỳ vọng lý thuyết	PASS
2	`test_bmc03_witness_matches_c_runtime_bug`	Xác nhận nhân chứng BMC-03 làm chương trình C crash thực tế	PASS
3	`test_c_safe_full_scan_no_crash`	Xác nhận bản safe_full_scan chạy sạch, không lỗi ASan	PASS
4	`test_c_symbolic_safe_rejects_bad_input`	Xác nhận tiền điều kiện từ chối tham số xấu base_index=9	PASS
5	`test_c_symbolic_safe_runs_clean`	Xác nhận tiền điều kiện cho phép tham số hợp lệ base_index=3	PASS
6	`test_c_offbyone_detected`	Xác nhận lỗi lệch một BMC-02 bị ASan bắt chính xác	PASS
7	`test_c_reverse_scan_detected`	Xác nhận lỗi duyệt ngược BMC-05 bị ASan bắt tại bước đầu	PASS

Lệnh thực thi: `python tests/test_bmc.py` → Kết quả: **7/7 test đã PASS (100%)**.

9. TỔNG KẾT BÀI THỰC HÀNH VÀ BÀI HỌC KINH NGHIỆM

1. Làm chủ kỹ thuật Bounded Model Checking (BMC), thấu hiểu quy trình mở cuộn vòng lặp và phương pháp chuyển đổi bài toán kiểm tra an toàn thành bài toán giải thỏa mãn SMT.
2. Hiểu sâu sắc ý nghĩa của nhân chứng (witness): một công cụ kiểm chứng hình thức mạnh mẽ không chỉ phát hiện sự tồn tại của lỗi mà còn chỉ ra chính xác bộ dữ liệu kích hoạt lỗi để hỗ trợ tái hiện và vá lỗi tức thì.
3. Phân biệt rõ ranh giới giữa kiểm chứng có giới hạn và an toàn tuyệt đối, nắm vững kỹ thuật thiết lập tiền điều kiện để chủ động triệt tiêu lỗi bộ nhớ ngay từ cửa ngõ hệ thống.
4. Thấy rõ tiềm năng ứng dụng to lớn của BMC trong công nghiệp phát triển phần mềm an toàn hiện đại (như CBMC trong kiểm thử nhân Linux hoặc chứng minh an toàn vi xử lý).